

Bytes, Unicode, and Strawberry

Why a model can't see the letters it writes · video 2:59

This lesson answers one question: why is counting the letters in “strawberry” hard for a language model? Text reaches the model as **tokens** built from **UTF-8 bytes**, not as letters. That explains why every language can be read, why some languages cost several times more tokens, and why spelling and letter counting are hard: the letters are hidden inside tokens.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Code points and UTF-8	▶ 0:08–0:59	3	☐
2. Languages don't cost the same	▶ 0:59–1:26	3	☐
3. Strawberry: letters hidden in tokens	▶ 1:26–2:06	2	☐
4. Spelling it out, and what to do	▶ 2:06–2:57	2	☐

Before you start: Deep Dive episode 1 (byte-pair encoding). Code: `code/d02_bytes_unicode` (downloads two tokenizers and Qwen2.5-1.5B-Instruct, about 3 GB).

CONCEPT 1 OF 4

Code points and UTF-8

▶ Video 0:08-0:59

Unicode gives every character a number, its **code point**: “a” is 97 (U+0061), “é” U+00E9, “ж” U+0436, “中” U+4E2D, 🍓 U+1F353; more than a million are possible. **UTF-8** writes each code point as one to four bytes: plain English letters take 1 byte, accented and Russian letters 2, Chinese characters 3, emoji 4.

Unicode: every character has a number (a code point)

a	é	ж	中	🍓
U+0061	U+00E9	U+0436	U+4E2D	U+1F353
= 97				

more than a million possible code points

UTF-8: one to four bytes per character

a	01100001	1 byte
é	11000011 10101001	2 bytes
ж	11010000 10110110	2 bytes
中	11100100 10111000 10101101	3 bytes
🍓	11110000 10011111 10001101 10010011	4 bytes

yellow bits: how many bytes belong together
so 256 byte tokens can spell any language

▶ 0:35 Unicode gives every character a number: a code point.

▶ 0:58 UTF-8 writes code points as 1 to 4 bytes; the leading bits (yellow) say how many bytes belong together.

The first bits of each byte say how many bytes belong together (0... one byte, 110... the start of two, 1110... of three, 11110... of four, and 10... a continuation). That is why a byte-level tokenizer, with just 256 starting tokens, can read any language.

KEY IDEA

Characters → code points → UTF-8 bytes → tokens. The model only ever sees the last step.

Check 1: Code points and UTF-8

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many UTF-8 bytes is the word “café”?

1.2 MULTIPLE CHOICE

A byte starts with the bits 1110. What does it tell a UTF-8 reader?

- A. It is a plain ASCII character
- B. It starts a 3-byte character
- C. It continues the previous character
- D. It starts a 4-byte character

1.3 CALCULATION

The euro sign € is U+20AC. How many UTF-8 bytes does it need?

Languages don't cost the same

▶ Video 0:59-1:26

One sentence about a cat, in five languages. For **GPT-2**, English is **10** tokens; Russian is **38** and Chinese **25**, for the same meaning, because GPT-2 learned its merges mostly from English text. **Qwen2.5**, trained on far more languages, needs **16** tokens for the Russian and only **8** for the Chinese.

one sentence, five languages: how many tokens?				
language		bytes	GPT-2	Qwen2.5
English	The cat is sleeping on the warm windowsill.	43	10	10
Italian	Il gatto dorme sul davanzale caldo.	35	13	12
Russian	Кошка спит на тёплом подоконнике.	61	38	16
Chinese	猫在温暖的窗台上睡觉。	33	25	8
Emoji	🐱💤🪟☀️	18	12	7

GPT2 learned its merges mostly from English
same meaning, very different price and context use

▶ 1:25 One sentence in five languages: bytes and tokens for GPT-2 and Qwen2.5.

KEY IDEA

The tokenizer's training data sets the price of each language: in cost, speed and how much fits in the context window.

Check 2: Languages don't cost the same

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

For GPT-2, how many times more tokens does the Russian sentence need than the English one?

.....

2.2 TRUE OR FALSE

"The Chinese sentence is 11 characters but 33 bytes."

True ☐ False ☐

2.3 SHORT ANSWER

A product charges per token. Why might Russian-speaking users pay more with a GPT-2-style tokenizer?

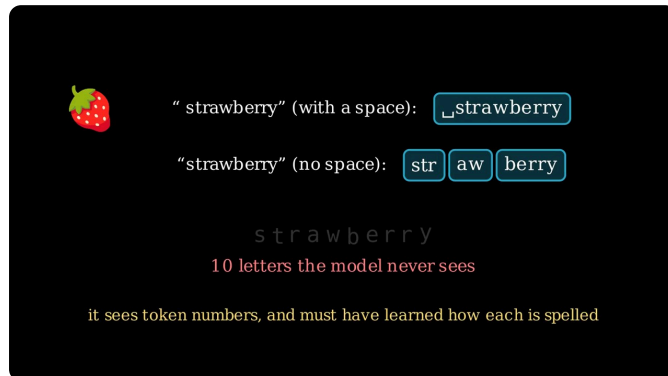
.....

CONCEPT 3 OF 4

Strawberry: letters hidden in tokens

▶ Video 1:26–2:06

With a space in front, as it usually appears in a sentence, “strawberry” is a **single token**; without the space, three: **str** | **aw** | **berry**. Either way, the model never sees the ten letters. It sees one or three token numbers, and has to have learned how each one is spelled.



“strawberry” (with a space): `▮strawberry`

“strawberry” (no space): `str` `aw` `berry`

strawberry
10 letters the model never sees

it sees token numbers, and must have learned how each is spelled

▶ 1:43 “strawberry” is one token; without the space, str | aw | berry. The ten letters are never seen.

Qwen2.5-1.5B-Instruct: count the letter

strawberry, letter r	true 3	model 3 ✓	<code>▮strawberry</code>
bookkeeper, letter e	true 3	model 3 ✓	<code>▮book</code> <code>keeper</code>
mississippi, letter s	true 4	model 4 ✓	<code>▮miss</code> <code>issippi</code>
nevertheless, letter e	true 4	model 3 ✗	<code>▮nevertheless</code>

famous question: possibly memorised

▶ 2:05 Counting letters: three of four right; nevertheless, just two tokens, gets 3 instead of 4.

Asked directly, Qwen2.5-1.5B says strawberry has 3 r's: correct, but the question is famous and may simply have been learned. Bookkeeper and Mississippi were right too; but “nevertheless” got 3 e's instead of 4. To the model, nevertheless is just two tokens: never, theless.

KEY IDEA

A right answer to a famous question is not proof of a skill: test on cases the model cannot have memorised.

Check 3: Strawberry: letters hidden in tokens

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

How many letters are hidden inside the single token “▮strawberry” (not counting the space)?

.....

3.2 SHORT ANSWER

Why test “nevertheless” and “bookkeeper” instead of only “strawberry”?

.....

CONCEPT 4 OF 4

Spelling it out, and what to do

▶ Video 2:06-2:57

Asked to spell strawberry first, the model wrote *s-t-r-o-w-a-b-e*: it misspelled the word and counted one r. Given the letters with spaces, so every letter is its own token, it said **four**. Seeing the letters is necessary, but a small model still struggles to count them.

"Spell the word strawberry letter by letter, then count the r's."

Sure! Here is the spelling of "strawberry" letter by letter:

s - t - r - o - w - a - b - e

... There is 1 'r' at the beginning.

misspelled: s-t-r-o-w-a-b-e, and counted one r

one token per letter

s t r o w a b e

answer: 4 x

seeing the letters is necessary, not sufficient

▶ 2:24 Asked to spell first, the model misspelled strawberry and counted one r; with one token per letter, it said four.

tokens hide letters, so these are hard:

spelling counting letters rhyming

reversing a word odd number chunks

fixes: call a tool (a short piece of code) a larger model check

▶ 2:41 Tasks that tokens make hard, and the fixes.

Because tokens hide letters, **spelling**, **counting characters**, **rhyming** and **reversing** a word are surprisingly hard, and numbers can be split into odd chunks too. The fixes: let the model **call a tool** (a short piece of code, LLMs in Practice episode 7), use a **larger model**, and **check**.

KEY IDEA

For exact character-level work, give the model code to run instead of asking it to look.

Check 4: Spelling it out, and what to do

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Which task is easiest for a token-based model?

- A. Reverse the letters of "strawberry"
- B. Count the e's in "nevertheless"
- C. Translate "strawberry" into Italian
- D. List the letters of "bookkeeper" in order

4.2 HANDS-ON CODE

Try it yourself. Add this to `bytes_unicode.py`: how does Qwen2.5 tokenize these words, and which do you expect the model to find hardest to spell?

```
for w in ["strawberry", "unbelievable", "Mississippi", "hippopotamus"]:
    print(w, [qwen.decode([i]) for i in qwen(w)["input_ids"]])
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Code points and UTF-8 · rewatch ▶ 0:08-0:59

1.1 5. c, a, f are 1 byte each; é is 2 bytes.

1.2 B. 1110 = three leading ones: this byte and two continuation bytes (10...) form one character, like 中.

1.3 3. Code points from U+0800 to U+FFFF use 3 bytes (checked: '€'.encode() has length 3).

2. Languages don't cost the same · rewatch ▶ 0:59-1:26

2.1 3.8 times. $38 / 10 = 3.8$.

2.2 True. Each Chinese character (and the full stop 。) takes 3 bytes: $11 \times 3 = 33$.

2.3 The same text needs far more tokens (38 vs 10 for our sentence), because the tokenizer has few merges for Cyrillic. Fewer merges learned for a script means shorter tokens and longer sequences.

3. Strawberry: letters hidden in tokens · rewatch ▶ 1:26-2:06

3.1 10. s-t-r-a-w-b-e-r-r-y.

3.2 Strawberry is a famous test the model may have memorised; less famous words check whether it can actually count. A memorised answer and a real skill look the same on a famous question.

4. Spelling it out, and what to do · rewatch ▶ 2:06-2:57

4.1 C. Translation works at the level of words and meaning; the others need the hidden letters.

4.2 “strawberry”, “unbelievable” and “Mississippi” are each one token; “hippopotamus” is “hipp | opot | amus”. The single-token words hide all their letters, so expect them to be the hardest to spell or count letter by letter. The fewer and longer the tokens, the more the model must have memorised about their spelling.